

```
import numpy as np
import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

```
# printing the stopwords in English
print(stopwords.words('english'))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'a
```

```
# Load datasets
true_news = pd.read_csv('True.csv')
fake_news = pd.read_csv('Fake.csv')
```

```
# Add labels
true_news['label'] = 0
fake_news['label'] = 1
```

```
# Combine datasets
news_dataset = pd.concat([true_news, fake_news], axis=0)
```

```
# Shuffle dataset
news_dataset = news_dataset.sample(frac=1, random_state=42)
```

```
# Reset index
news_dataset.reset_index(drop=True, inplace=True)
```

```
# Display first rows
print(news_dataset.head())
```

```

                                title \
0  BREAKING: GOP Chairman Grassley Has Had Enoug...
1  Failed GOP Candidates Remembered In Hilarious...
2  Mike Pence's New DC Neighbors Are HILARIOUSLY...
3  California AG pledges to defend birth control ...
4  AZ RANCHERS Living On US-Mexico Border Destroy...

                                text      subject \
0  Donald Trump s White House is in chaos, and th...      News
1  Now that Donald Trump is the presumptive GOP n...      News
2  Mike Pence is a huge homophobe. He supports ex...      News
3  SAN FRANCISCO (Reuters) - California Attorney ...  politicsNews
4  Twisted reasoning is all that comes from Pelos...    politics

      date  label
0  July 21, 2017    1
1    May 7, 2016    1
2  December 3, 2016    1
3  October 6, 2017    0
4    Apr 25, 2017    1
```

Start coding or [generate](#) with AI.

```
#Text Preprocessing
# Combine title and text
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
nltk.download('stopwords')

port_stem = PorterStemmer() #initialize the stemmer
stop_words = set(nltk.corpus.stopwords.words('english')) #convert stopwords to a set outside the function for o(1) lookups

def stemming(content):
    review = re.sub('[^a-zA-Z]', ' ', str(content))
    review = review.lower()
    words = review.split()

    stemmed_words = [port_stem.stem(word) for word in words if word not in stop_words]

    return ' '.join(stemmed_words)

news_dataset['content'] = news_dataset['title'] + ' ' + news_dataset['text']

news_dataset['content'] = news_dataset['content'].apply(stemming)

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
#Features and Labels
X = news_dataset['content']
Y = news_dataset['label']
```

```
#TF-IDF Vectorization
vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(X)
```

```
#Train-Test Split
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42,
    stratify=Y
)
```

```
#Logistic Regression Model
model = LogisticRegression(max_iter=1000)

model.fit(X_train, Y_train)
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression(max_iter=1000)
```

```
#Accuracy
# Training Accuracy

train_prediction = model.predict(X_train)

train_accuracy = accuracy_score(Y_train, train_prediction)

print("Training Accuracy:", train_accuracy)

# Testing Accuracy

test_prediction = model.predict(X_test)

test_accuracy = accuracy_score(Y_test, test_prediction)
```

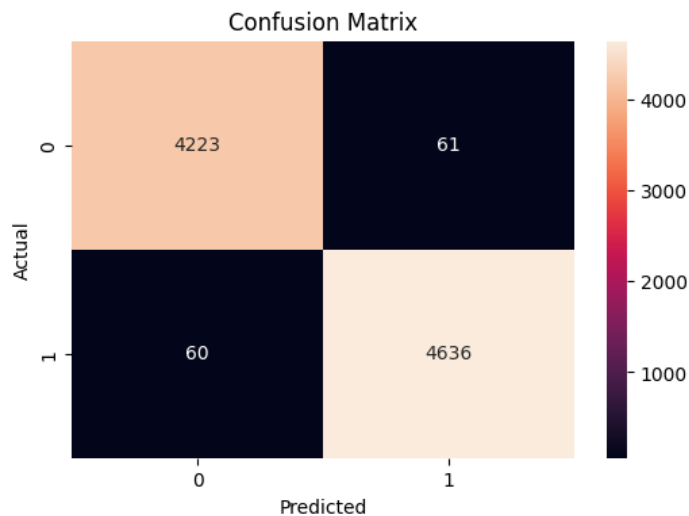
```
print("Testing Accuracy:", test_accuracy)
```

```
Training Accuracy: 0.9915641182693914
Testing Accuracy: 0.9865256124721603
```

```
# Confusion Matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

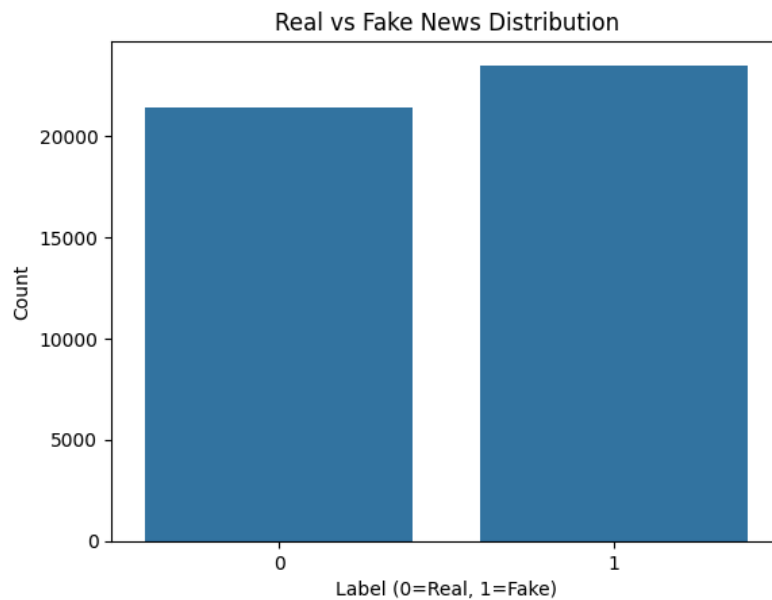
cm = confusion_matrix(Y_test, test_prediction)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



```
#class Distribution(Shows how many Real vs Fake news articles are in the dataset.)
import seaborn as sns
import matplotlib.pyplot as plt

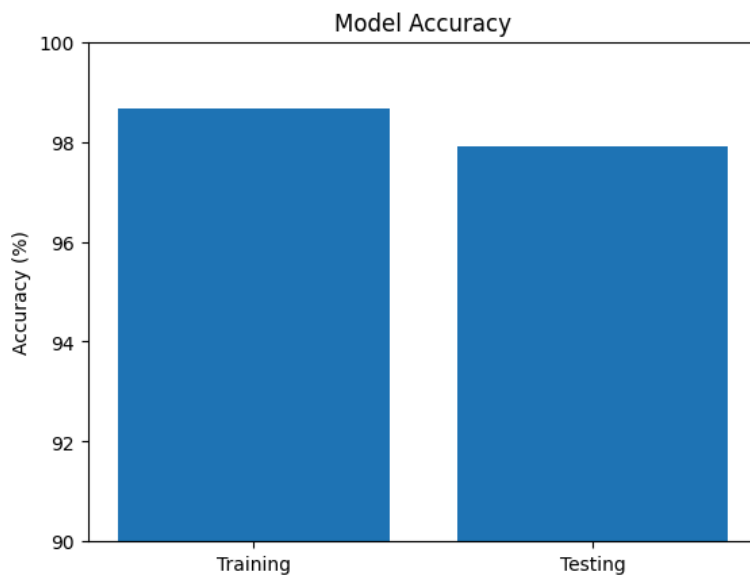
sns.countplot(x='label', data=news_dataset)
plt.title('Real vs Fake News Distribution')
plt.xlabel('Label (0=Real, 1=Fake)')
plt.ylabel('Count')
plt.show()
```



```
#Accuracy Comparison(Useful for showing Training vs Testing Accuracy.)
#Model Accuracy(Comparison of training and testing accuracy.)
import matplotlib.pyplot as plt

labels = ['Training', 'Testing']
accuracy = [98.66, 97.91]

plt.bar(labels, accuracy)
plt.title('Model Accuracy')
plt.ylabel('Accuracy (%)')
plt.ylim(90,100)
plt.show()
```



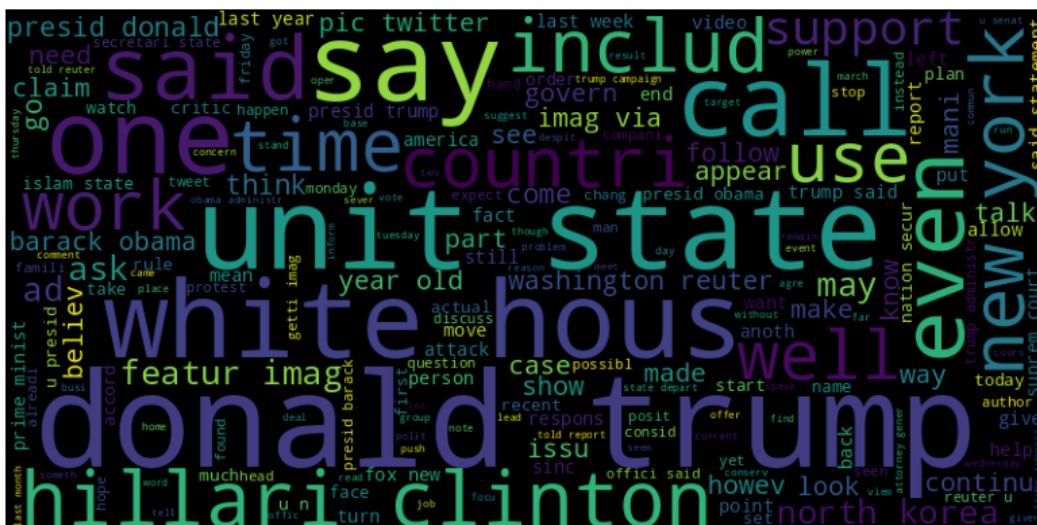
```
#Word Cloud (Shows the most common words in news articles.)
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text = " ".join(news_dataset['content'])

wordcloud = WordCloud(width=800, height=400).generate(text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis('off')
plt.title('Most Common Words')
plt.show()
```

Most Common Words



```
#Predictive System
news = X_test[0]

prediction = model.predict(news)

if prediction[0] == 0:
    print("Real News")
else:
    print("Fake News")
```

Fake News

```
#prediction for user input
user_news = input("Enter News Text: ")

user_news = stemming(user_news)

user_news_vector = vectorizer.transform([user_news])

prediction = model.predict(user_news_vector)

if prediction[0] == 0:
    print("📰 Real News")
else:
    print("💣 Fake News")
```